

# Agentic BI

Groq API · LLaMA 3.3-70B Versatile

Natural Language → SQL → Insight

Text-based SQL Pipeline · DuckDB · Gold Iceberg · Streamlit

| Thành phần      | Chi tiết                                                                 |
|-----------------|--------------------------------------------------------------------------|
| LLM Provider    | <b>Groq API</b> — LPU inference, sub-second latency                      |
| Model           | <code>llama-3.3-70b-versatile</code>                                     |
| Mode            | Text-based SQL (JSON output, self-correction retries)                    |
| Query Engine    | DuckDB in-memory — đọc trực tiếp từ Gold Iceberg tables                  |
| Frontend        | Streamlit — chat UI + Plotly chart                                       |
| Intent Classes  | <code>DATA_QUERY</code> · <code>FOLLOWUP</code> · <code>SMALLTALK</code> |
| Self-correction | Tối đa 3 lần retry với error context                                     |
| Post-insight    | Groq gọi thêm 1 lần để sinh 2–3 câu business insight                     |

# Mục lục

|           |                                                                      |           |
|-----------|----------------------------------------------------------------------|-----------|
| <b>1</b>  | <b>Tổng quan — Agentic BI với Groq</b>                               | <b>2</b>  |
| 1.1       | Vị trí trong hệ thống . . . . .                                      | 2         |
| 1.2       | Tại sao Groq? . . . . .                                              | 2         |
| <b>2</b>  | <b>Kiến trúc Pipeline — 4 Phase</b>                                  | <b>3</b>  |
| <b>3</b>  | <b>Phase 1 — Intent Classification</b>                               | <b>4</b>  |
| <b>4</b>  | <b>Phase 2 — SQL Generation (Groq LLaMA 3.3-70B)</b>                 | <b>6</b>  |
| <b>5</b>  | <b>Phase 3 — DuckDB Execution &amp; Self-correction</b>              | <b>7</b>  |
| <b>6</b>  | <b>Phase 4 — Business Insight Summarization</b>                      | <b>8</b>  |
| <b>7</b>  | <b>Script Demo Chi Tiết</b>                                          | <b>9</b>  |
| <b>8</b>  | <b>Sơ đồ Chi Tiết — Groq Text-based vs Claude Tool Use</b>           | <b>11</b> |
| <b>9</b>  | <b>Code Reference — Đường đi thực tế</b>                             | <b>12</b> |
| 9.1       | Config — Chọn provider . . . . .                                     | 12        |
| 9.2       | <code>_call_llm</code> — HTTP call tới Groq . . . . .                | 12        |
| 9.3       | <code>_generate_text_based</code> — SQL pipeline với retry . . . . . | 13        |
| 9.4       | Streamlit UI — Status và render . . . . .                            | 13        |
| <b>10</b> | <b>Sơ đồ Đây Đủ — Groq Agentic BI End-to-End</b>                     | <b>15</b> |
| <b>11</b> | <b>Demo Questions — Cheat Sheet</b>                                  | <b>16</b> |
| <b>12</b> | <b>Phụ lục — Q&amp;A về Agentic BI</b>                               | <b>18</b> |

# 1 Tổng quan — Agentic BI với Groq

## 1.1 Vị trí trong hệ thống

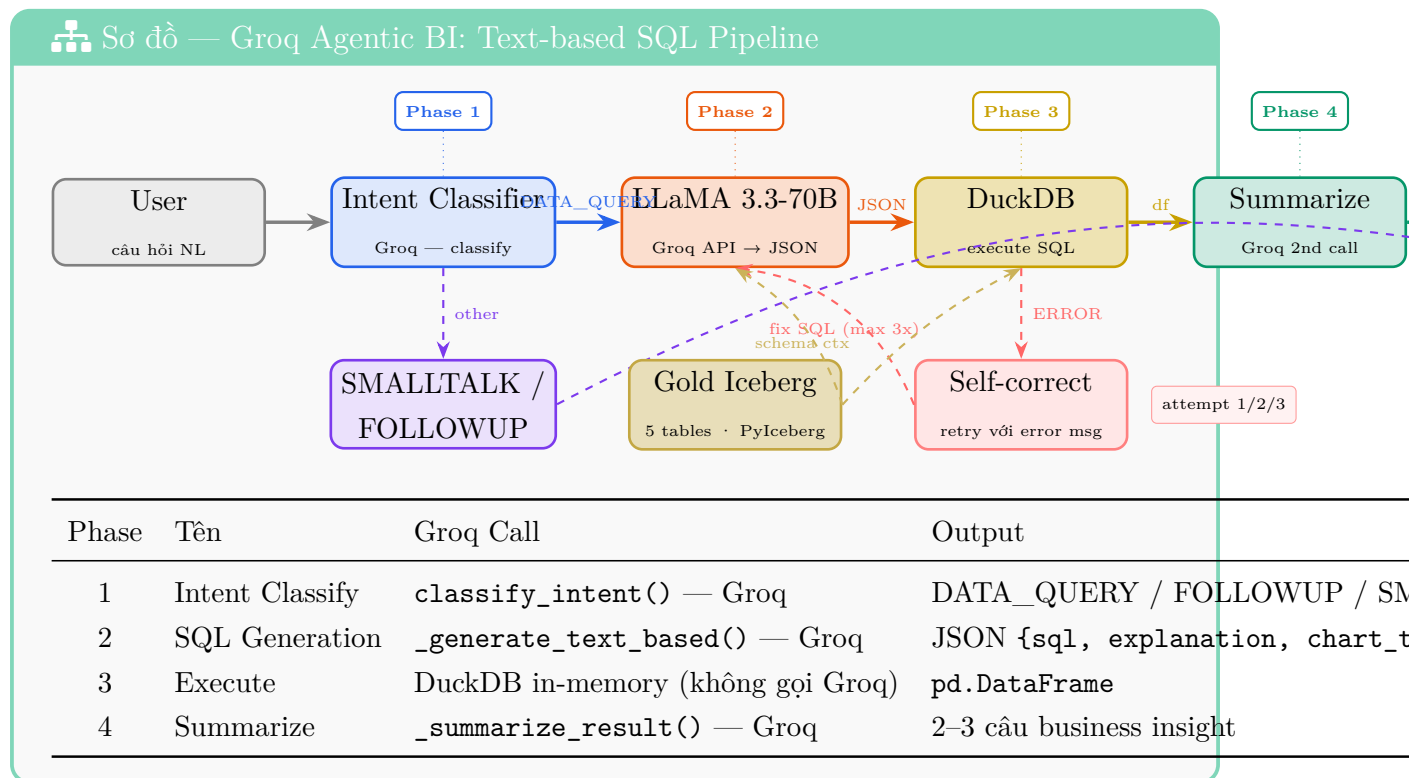
Agentic BI là **layer cuối cùng** trong Medallion pipeline — nhận Gold tables (đã aggregate và partitioned) và cho phép người dùng hỏi bằng ngôn ngữ tự nhiên thay vì viết SQL thủ công.



## 1.2 Tại sao Groq?

| Điểm mạnh                | Lý do chọn                                                                                                                           |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <b>Tốc độ siêu nhanh</b> | Groq LPU (Language Processing Unit) — không phải GPU, thiết kế chuyên biệt cho inference. Output token: >500 tok/s vs GPU ~60 tok/s. |
| <b>Free tier đủ dùng</b> | 6000 req/ngày, 500K token/phút — không tốn chi phí cho demo và prototype.                                                            |
| <b>LLaMA 3.3-70B</b>     | Open-weight model Meta, hỗ trợ tiếng Việt tốt, giải sinh SQL, context 128K token.                                                    |
| <b>OpenAI-compatible</b> | API <code>/v1/chat/completions</code> — dùng chung <code>requests.post</code> , không cần SDK riêng.                                 |

## 2 Kiến trúc Pipeline — 4 Phase



### 3 Phase 1 — Intent Classification

#### Phase 1: Intent Classification

Trước khi gọi SQL pipeline, hệ thống phân loại câu hỏi thành một trong ba loại:

| Intent     | Xử lý                                              | Ví dụ                      |
|------------|----------------------------------------------------|----------------------------|
| DATA_QUERY | Chạy full SQL pipeline (Phase 2 → 4)               | “Doanh thu theo tháng?”    |
| FOLLOWUP   | Groq trả lời dựa trên lịch sử chat, không query DB | “Sao bang SP lại cao vậy?” |
| SMALLTALK  | Groq trả lời trực tiếp, không query DB             | “Bạn là AI gì?”            |

**Cách hoạt động:** `classify_intent()` gọi Groq với system prompt mô tả ba intent và rules. LLaMA 3.3-70B trả về một từ: `DATA_QUERY`, `FOLLOWUP`, hoặc `SMALLTALK`. Nếu parse lỗi, mặc định fallback về `DATA_QUERY`.



## 4 Phase 2 — SQL Generation (Groq LLaMA 3.3-70B)

### ⚡ Phase 2: SQL Generation — Text-based JSON

#### System Prompt Strategy

Thay vì dùng native **Tool Use API** (như Anthropic), Groq path dùng **text-based prompting**: yêu cầu LLM trả về *thuần JSON*, sau đó parse bằng Python.

System prompt bao gồm:

- Luật SQL: chỉ SELECT, tự thêm LIMIT 1000, tên bảng available
- Schema đầy đủ 5 Gold tables (tên cột, kiểu dữ liệu) — inject lúc runtime
- Quy tắc chart type: bar/line/pie/scatter/none
- **6 few-shot examples** (Q/A pair) — giúp LLM học pattern SQL chuẩn
- Luật đặc biệt cho sentiment table (`review_sentiment`) — không dùng `review_score` để suy ra sentiment
- Format output: JSON thuần, không markdown, không dấu backtick

Output format mong đợi:

#### JSON output từ LLaMA 3.3-70B

```
{
  "sql": "SELECT customer_state, COUNT(*) AS orders ...",
  "explanation": "Top 10 bang có nhiều đơn hàng nhất.",
  "chart_type": "bar"
}
```

#### Few-shot Examples (trích từ code)

##### Ví dụ few-shot trong system prompt

Q: Doanh thu theo tháng năm 2018?

A: {"sql": "SELECT strftime(purchased\_at, '%Y-%m') AS month, ROUND(SUM(payment\_value),2) AS revenue\_brl FROM fct\_orders WHERE purchased\_at >= '2018-01-01' AND purchased\_at < '2019-01-01' GROUP BY 1 ORDER BY 1", "explanation": "Tổng doanh thu BRL theo tháng 2018.", "chart\_type": "line"}

Q: Tỷ lệ các loại thanh toán?

A: {"sql": "SELECT payment\_type, COUNT(\*) AS cnt FROM fct\_orders WHERE payment\_type IS NOT NULL GROUP BY payment\_type ORDER BY cnt", "explanation": "Số đơn hàng theo phương thức thanh toán.", "chart\_type": "pie"}

## 5 Phase 3 — DuckDB Execution & Self-correction

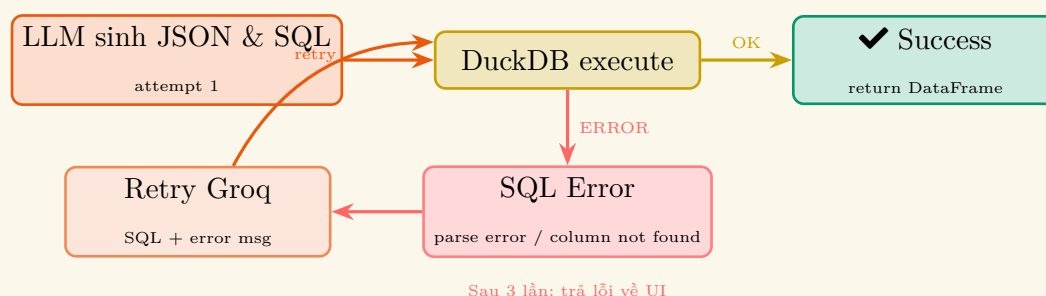
### Phase 3: Execute + Self-correction Loop

#### DuckDB In-memory Views

DuckDB đọc dữ liệu trực tiếp từ Gold Iceberg tables thông qua PyIceberg:

1. PyIceberg load 5 Gold tables → Arrow format
2. Đăng ký mỗi table thành **DuckDB in-memory view** (`con.register("fct_orders", df)`)
3. SQL từ LLaMA chạy trực tiếp trên views — không cần Spark, không cần Trino
4. Kết quả trả về `pd.DataFrame` trong vài milliseconds

#### Self-correction Loop (tối đa 3 lần)



Sau 3 lần: trả lỗi về UI

Fix prompt khi SQL lỗi:

#### Self-correction prompt gửi cho Groq

SQL sau gặp lỗi khi thực thi trên DuckDB:

SQL: `SELECT seller_id, AVG(review_score) ...`

Lỗi: `Binder Error: "review_score" not found. Did you mean "avg_review_score"`

Hãy sửa lại SQL. Chỉ trả về JSON (không markdown):

`{"sql": "...", "explanation": "...", "chart_type": "..."}"`



## 6 Phase 4 — Business Insight Summarization

### 💡 Phase 4: Summarize Result

Sau khi DuckDB trả về DataFrame, Groq được gọi **lần thứ hai** để sinh 2–3 câu business insight:

- Input: câu hỏi gốc + top 10 dòng đầu từ DataFrame
- Output: 2–3 câu tiếng Việt, nêu số liệu cụ thể với đơn vị đầy đủ
- Quy tắc: không dùng “có thể”, “dường như”, “khoảng” — chỉ fact cứng
- Tiền BRL: format “X,XX tỷ R\$” / “X,XX triệu R\$”
- Highlight giá trị cao nhất / thấp nhất / bất thường

**Ví dụ output:**

*“São Paulo dẫn đầu với 41.746 đơn hàng, chiếm 41,6% tổng số đơn toàn quốc. Rio de Janeiro đứng thứ 2 với 12.852 đơn — cách biệt gần 3,3x so với SP. Hai bang này cộng lại đã chiếm hơn 54% thị phần đơn hàng Olist.”*

## 7 Script Demo Chi Tiết

### Người 4 · Phần Agentic BI (khoảng 2 phút)

#### [Mở Streamlit Agentic BI]

[11:30 – 13:30]

##### DEMO — Thao tác màn hình

##### Chuẩn bị:

- Đảm bảo `AI_PROVIDER=groq` và `GROQ_API_KEY` đã set trong `.env`
- Mở `http://localhost:8501` → chọn page **Olist Agentic BI**
- Sidebar sẽ hiển thị: LLM: `llama-3.3-70b-versatile` và Mode: `Text-based SQL`
- 5 Gold tables đều phải có dấu  trong sidebar

#### Demo 1 — Câu hỏi đơn: Top doanh thu theo bang

[11:30 – 12:10]

##### DEMO — Thao tác màn hình

##### Gõ vào chat input:

*“Top 10 bang có doanh thu cao nhất năm 2018?”*

##### Quan sát:

1. Spinner “Đang xử lý...” hiện ra → “Phân tích câu hỏi và sinh SQL...”
2. Sau ~1 giây (Groq rất nhanh): tag `DATA QUERY` xanh lam xuất hiện
3. Click expand **SQL** để show câu query được sinh ra
4. Bar chart xuất hiện bên dưới
5. Insight text: 2–3 câu về doanh thu SP, RJ, MG

“Đây là **Agentic BI** — chat interface để hỏi dữ liệu Olist bằng tiếng Việt thay vì viết SQL.

Backend dùng **Groq API** với model **LLaMA 3.3-70B** — open-weight model của Meta, chạy trên phần cứng LPU của Groq, nhanh hơn GPU thông thường từ 5–10 lần.

(expand SQL) Nhìn vào đây — LLM tự sinh câu `GROUP BY`, tự filter năm 2018, tự `ORDER BY` revenue, tự chọn bar chart vì đây là bài toán ranking. Nhóm em không hard-code gì — toàn bộ logic SQL đến từ model.

Kết quả: São Paulo dẫn đầu xa — gấp hơn 3 lần Rio de Janeiro. Đây là insight business thực tế từ 100 nghìn đơn hàng thật.”

## Demo 2 — So sánh tỷ lệ giao trễ

[12:10 – 12:50]

### DEMO — Thao tác màn hình

#### Gõ tiếp vào chat:

*“So sánh tỷ lệ giao trễ giữa các phương thức thanh toán”*

#### Quan sát:

1. SQL sinh ra sẽ có `CASE WHEN delivery_status='late' THEN 1 ELSE 0`  
`END`
2. Chart type: bar (so sánh danh mục)
3. Insight: phương thức nào có tỷ lệ trễ cao nhất?

“Câu hỏi lần này phức tạp hơn — cần tính tỷ lệ phần trăm. LLM tự sinh `CASE WHEN` để tính late orders, rồi chia cho total để ra tỷ lệ.

Đây là điểm quan trọng: người dùng không cần biết schema, không cần biết `delivery_status='late'` là cột gì hay bảng nào. LLM đọc schema context và tự điền vào.

Nếu SQL sai — ví dụ tên cột không đúng — hệ thống **tự retry** tối đa 3 lần, mỗi lần gửi lại error message cho Groq để sửa. Người dùng không thấy gì — chỉ thấy kết quả cuối cùng.”

## Demo 3 — Follow-up câu hỏi

[12:50 – 13:30]

### DEMO — Thao tác màn hình

#### Gõ tiếp (không cần context mới):

*“Phương thức nào đáng lo ngại nhất? Tại sao?”*

#### Quan sát:

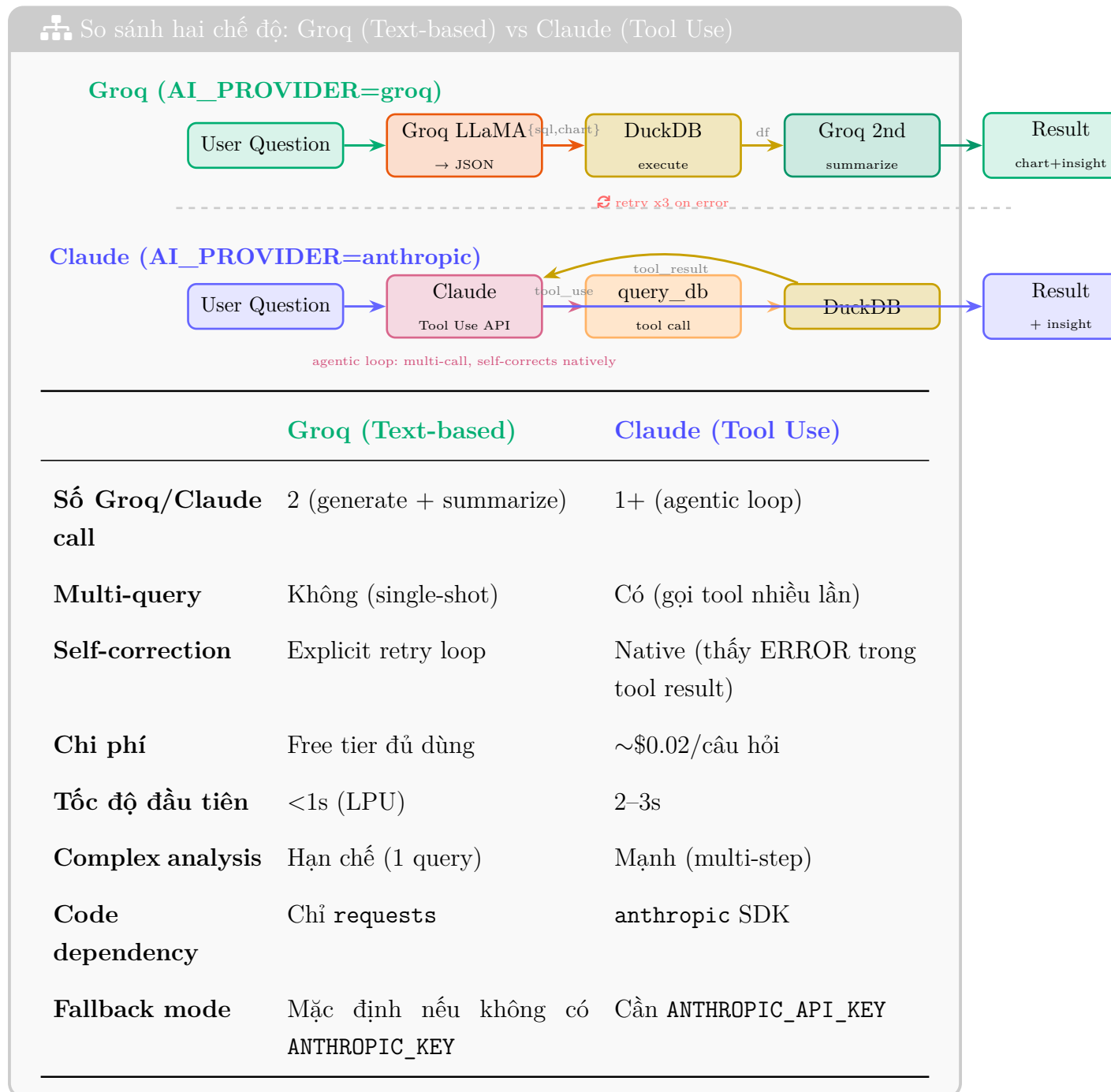
1. Tag FOLLOW-UP màu xanh lá xuất hiện (không phải DATA QUERY)
2. **Không có SQL** — Groq trả lời dựa trên kết quả câu trước trong history
3. Câu trả lời phân tích narrative từ data đã có

“Câu này không có SQL mới — hệ thống nhận diện đây là **FOLLOWUP** intent, chỉ cần diễn giải data cũ.

Groq đọc lịch sử 10 tin nhắn gần nhất và trả lời như một analyst — nêu cụ thể phương thức nào và con số tỷ lệ, không bịa số mới.

Ba intent class này — `DATA_QUERY`, `FOLLOWUP`, `SMALLTALK` — cho phép hệ thống **không lãng phí** Groq call vào database khi không cần thiết, đồng thời duy trì context conversation tự nhiên.”

## 8 Sơ đồ Chi Tiết — Groq Text-based vs Claude Tool Use



## 9 Code Reference — Đường đi thực tế

### 9.1 Config — Chọn provider

#### bi/config.py — Groq settings

```
# .env
AI_PROVIDER = groq
GROQ_API_KEY = gsk_XXXXXXXXXXXXXXXXXXXXX
GROQ_MODEL   = llama-3.3-70b-versatile   # default

# config.py reads env and exposes:
AI_PROVIDER  = "groq"
GROQ_KEY     = "gsk_..."
GROQ_MODEL   = "llama-3.3-70b-versatile"
MAX_ROWS     = 1000
MAX_RETRIES  = 3
```

### 9.2 \_\_call\_\_llm — HTTP call tới Groq

#### bi/agent.py — Groq API call (OpenAI-compatible endpoint)

```
def _call_llm(messages, system, max_tokens=1024):
    # Groq path
    url    = "https://api.groq.com/openai/v1/chat/completions"
    model  = GROQ_MODEL # "llama-3.3-70b-versatile"
    key    = GROQ_KEY

    all_msgs = [{"role": "system", "content": system}] + messages
    resp = requests.post(
        url,
        headers={"Authorization": f"Bearer {key}"},
        json={"model": model, "messages": all_msgs,
              "max_tokens": max_tokens, "temperature": 0},
        timeout=60,
    )
    return resp.json()["choices"][0]["message"]["content"]
```

### 9.3 \_\_generate\_\_text\_\_based — SQL pipeline với retry

#### bi/agent.py — Text-based SQL + self-correction

```
def _generate_text_based(question, history, schema, con):
    system = _SYSTEM_SQL.format(
        max_rows=MAX_ROWS,
        table_names=", ".join(GOLD_TABLES),
        schema=schema,
    )
    base_msgs = history[-MAX_HISTORY:] + [{"role": "user", "content": question}]

    for attempt in range(MAX_RETRIES):  # tối đa 3 lần
        if attempt == 0:
            msgs = base_msgs
        else:
            # Kèm SQL lỗi + error message → Groq tự fix
            msgs = base_msgs + [
                {"role": "assistant", "content": prev_raw},
                {"role": "user", "content": FIX_MSG.format(
                    sql=raw_sql, error=last_error)}
            ]

        raw = _call_llm(msgs, system)  # Groq call
        parsed = _parse_json(raw)  # Extract JSON
        sql = parsed["sql"] + " LIMIT 1000"
        df = con.execute(sql).df()  # DuckDB execute
        return {**parsed, "result_df": df}  # success!
    # on Exception: last_error = str(e), retry
```

### 9.4 Streamlit UI — Status và render

#### bi/agentic\_bi.py — Streamlit processing

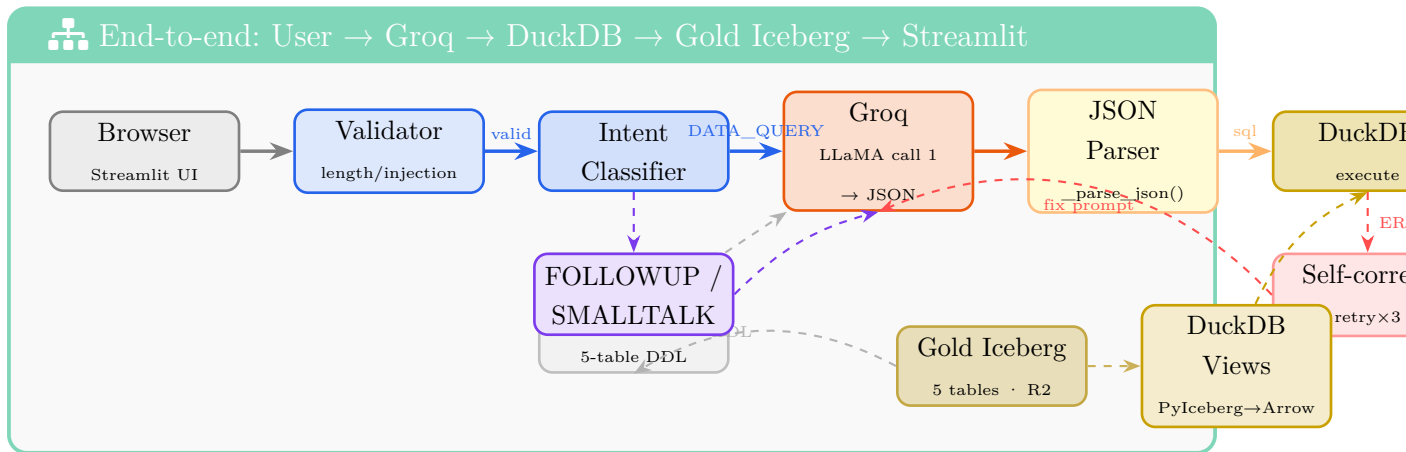
```
with st.status("Đang xử lý...", expanded=True) as status:
    if AI_PROVIDER == "groq":
        status.write("Phân tích câu hỏi và sinh SQL...")

    res = run(intent=intent, question=user_input,
              history=chat_history, schema=schema_ctx, con=con)

    status.update(label="Hoàn thành", state="complete")
```

```
# Render: intent tag + SQL expander + chart + insight box  
_render_assistant({...})
```

## 10 Sơ đồ Đây Đủ — Groq Agentic BI End-to-End





## 11 Demo Questions — Cheat Sheet

### ☰ Câu hỏi demo + Expected Output

| Câu hỏi                                            | Chart              | Expected Insight                               |
|----------------------------------------------------|--------------------|------------------------------------------------|
| Top 10 bang doanh thu cao nhất 2018?               | bar                | SP $\gg$ RJ, chiếm >40%                        |
| So sánh tỷ lệ giao trễ theo phương thức thanh toán | bar                | boleto có tỷ lệ trễ cao hơn credit_card        |
| Doanh thu theo tháng năm 2018?                     | line               | peak tháng 11 (Black Friday Brazil)            |
| Phân tích churn: % khách churned?                  | pie                | ~97% churned (1-time buyers)                   |
| Tỷ lệ sentiment của review?                        | pie                | positive $\approx$ 75%, negative $\approx$ 13% |
| Seller nào review tốt nhất >100 đơn?               | bar                | top sellers: avg score >4.8                    |
| Tỷ lệ chuyển đổi lead theo kênh?                   | bar                | organic_search vs paid                         |
| So sánh Q1/2017 vs Q1/2018                         | bar                | 2018 tăng mạnh, growth >100%                   |
| <b>Follow-up (sau câu query trước):</b>            |                    |                                                |
| "Sao SP lại cao vậy?"                              | none<br>(followup) | Groq phân tích từ data đã có                   |
| "Xu hướng gì đáng lo?"                             | none<br>(followup) | Groq highlight anomaly                         |

 Lưu ý khi demo

- **Groq free tier:** 30 req/min, 6000 req/day — đủ cho demo 15 phút
- **Lần đầu khởi động:** PyIceberg load 5 tables vào DuckDB mất ~10s (chỉ một lần duy nhất, sau đó cached)
- **Nếu câu hỏi quá phức tạp:** Groq text-based chỉ sinh 1 SQL. Demo nên dùng câu hỏi đơn rõ ràng, tránh “phân tích toàn diện...”
- **Sidebar check:** trước khi demo verify 5 bảng ✅: `fct_orders`, `fct_funnel`, `dim_sellers`, `dim_customers`, `review_sentiment`
- **Nếu SQL lỗi:** hệ thống tự retry, spinner hiển thị. Chờ thêm 2–3s.
- **Mode label:** sidebar hiển thị “Mode: Text-based SQL” — đây là Groq mode. Nếu thấy “Tool Use” nghĩa là đang dùng Claude mode.

## 12 Phụ lục — Q&A về Agentic BI

### ? Câu hỏi thường gặp — Agentic BI

| Câu hỏi                                               | Trả lời                                                                                                                                                                                                                           |
|-------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Sao không dùng Claude Tool Use?                       | Groq <b>miễn phí</b> và <b>nhANH hơn</b> cho single-query use case. Claude Tool Use mạnh hơn ở multi-step analysis nhưng tốn tiền. Hệ thống <b>hỗ trợ cả hai</b> — chỉ cần đổi <code>AI_PROVIDER</code> trong <code>.env</code> . |
| Tại sao không dùng text-to-SQL riêng (vd: Langchain)? | Không cần thêm dependency. Groq với system prompt + few-shot đủ mạnh, self-correction tự xử lý 95% SQL lỗi.                                                                                                                       |
| Có thể hỏi join nhiều bảng không?                     | Có. Schema context gồm cả 5 bảng. LLM có thể sinh JOIN. Ví dụ: “Seller state nào có review sentiment tốt nhất?” → <code>JOIN dim_sellers + review_sentiment</code> .                                                              |
| DuckDB có chậm không khi data lớn?                    | DuckDB in-memory với 100K rows Gold tables: <100ms. Bottleneck là Groq latency (~800ms) — nhưng vẫn nhanh hơn nhiều so với Spark hay Trino cho ad-hoc query nhỏ.                                                                  |
| SQL injection risk?                                   | System prompt enforce “chỉ SELECT”. Ngoài ra validator ở <code>bi/validator.py</code> chặn các pattern nguy hiểm trước khi gửi cho LLM.                                                                                           |
| Groq có hỗ trợ tiếng Việt tốt không?                  | LLaMA 3.3-70B hỗ trợ tiếng Việt tốt ở mức production. System prompt và few-shot examples đều tiếng Việt. Insight output bằng tiếng Việt, số liệu đúng đơn vị.                                                                     |